

# Spesifikasyon: Java 27 Öğrenim Müfredatı

**Durum:** Teslim edildi, sonra eklendi — aşağıdaki *Ek 1* bölümüne bakın **Özgün kapsam:** 24 görevin hepsi tamam, 2026-09-23'te doğrulandı **Tarih:** 2026-09-23 **Aşama:** Specify → Plan → Tasks → Implement dizisinin 1. aşaması (Specify)

## Yaptığım Varsayımlar

- Öğrenci başka bir dilde (Python, JavaScript, C#) zaten programlıyor ve **özellikle Java'ya** yeni. Modüller değişkenin ya da döngünün ne olduğunu yeniden anlatmıyor; Java'nın bunu nasıl yaptığını ve nerede ayrıştığını anlatıyor.
- Özgün istekteki "takeout" kelimesi **Özet Çıkarımlar** anlamına geliyor: her modülü kapatan kısa bir toparlama.
- İçerik ve tüm kod **İngilizce**. Bu spesifikasyon bir depo dokümanı, o da İngilizce.
- Müfredat yalnızca **çekirdek dili ve JDK standart kütüphanesini** kapsıyor. Spring yok, JDBC yok, build çatısı yok.
- Dersler ve örnek kodları **her modül klasörünün içinde birlikte** duruyor. Ödev çözümleri ise öğrenci kazara cevabı görmesin diye ayrı bir üst düzey `solutions/` ağacında.
- Build aracı yok. Örnekler tek dosya kaynak başlatma ile çalışıyor: `java Example.java`.

→ Bunlardan herhangi birini şimdi düzeltin, yoksa böyle devam ediyorum.

## Amaç

Başka bir dil bilen bir programcıyı alıp JDK 27 üzerinde modern Java'da akıcı kılan, kendi hızında ilerleyen bir Java müfredatı kurmak.

**Kullanıcı:** Genel programlama okuryazarlığı olan ve sıfır Java deneyimi olan bir geliştirici.

**Başarı neye benziyor:** Öğrenci numaralı modüllerde sırayla ilerliyor. Her modül bir alanı hafif ve sade bir tonda anlatıyor, yorumları sözdizimini tekrarlamak yerine *nedenini* öğretene çalıştırılabilir örneklerle destekliyor. Her modül Özet Çıkarımlar ve bir ödevle kapanıyor; referans çözümler var ama dersin dışında duruyor.

## Bu müfredatı şekillendiren bir bulgu

JDK 27, **15 Eylül 2026**'da genel kullanıma açıldı. **LTS değil** (güncel LTS JDK 25). Teslim ettiği JEP'ler neredeyse tamamen çalışma zamanı seviyesinde:

| JEP | Başlık                                                    | Durum               |
|-----|-----------------------------------------------------------|---------------------|
| 523 | Make G1 the Default Garbage Collector in All Environments | Final               |
| 527 | Post-Quantum Hybrid Key Exchange for TLS 1.3              | Final               |
| 534 | Compact Object Headers by Default                         | Final               |
| 536 | JFR In-Process Data Redaction                             | Final               |
| 531 | Lazy Constants                                            | Üçüncü Preview      |
| 532 | Primitive Types in Patterns, instanceof, and switch       | Beşinci Preview     |
| 533 | Structured Concurrency                                    | Yedinci Preview     |
| 537 | Vector API                                                | On İkinci İnkübatör |
| 538 | PEM Encodings of Cryptographic Objects                    | Üçüncü Preview      |

**Sonuç:** "Java 27 öğrenmek", dokuz yeni dil özelliği öğrenmek demek değil. O listedeki hiçbir şey yeni başlayanın kod yazma biçimini değiştirmiyor. Dürüst çerçeve, ve bu müfredatın benimsediği çerçeve şu: *JDK 27 çalışma ortamı ve araç zinciridir; öğrendiğiniz dil ise JDK 25 LTS'e kadar birikmiş kararlı Java'dır.* Kapanış modülü 26 ve 27'nin gerçekte neyi değiştirdiğini, LTS olmayan sürümlerin neden düşündürdüğünden az önem taşıdığını ve bir JEP'in nasıl okunacağını anlatıyor.

## Giriş kararı

JEP 512 (Compact Source Files and Instance Main Methods) **JDK 25'te kesinleşti**, dolayısıyla 27'de kararlı. Bu yüzden Modül 01 şöyle açılıyor:

```
void main() {  
    IO.println("Hello, World!");  
}
```

`public class` yok, `static` yok, `String[] args` yok, `import` yok. Modül 05 ise öğrenciyi bilinçli biçimde klasik forma "mezun ediyor", çünkü karşılaşacağı her gerçek kod tabanı, öğretici ve Stack Overflow cevabı onu kullanıyor. Kolay biçimi öğretip klasik olanı hiç açıklamamak, onu gerçek Java okuyamaz hâlde bırakırdı.

## Teknoloji Yığını

| Öğe           | Seçim                                                                                       |
|---------------|---------------------------------------------------------------------------------------------|
| JDK           | OpenJDK 27 (GA 2026-09-15)                                                                  |
| Dil seviyesi  | JDK 25'e kadar kararlı özellikler; preview özellikler yalnızca son modülde, açıkça etiketli |
| Build aracı   | Yok                                                                                         |
| Bağımlılıklar | Yok                                                                                         |
| Ders biçimi   | Markdown                                                                                    |
| Örnek biçimi  | Tek dosya <code>.java</code> , doğrudan çalıştırılabilir                                    |

## Proje Yapısı

|                                                                                                                                                                                                                                                                                     |                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Java-27-edu/<br>├─ README.md<br>├─ SPEC.md<br>├─ scripts/<br>│   └─ verify-examples.sh<br>bildirir<br>├─ modules/<br>│   ├── 01-getting-started/<br>│   │   ├── README.md<br>│   │   ├── examples/<br>│   │   └─ homework/<br>│   └─ ...<br>└─ solutions/<br>└─ 01-getting-started/ | → İçindekiler, nasıl başlanır, JDK kurulumu<br>→ Bu doküman<br>→ Her örneği derleyip çalıştırır, başarısızlıkları<br>→ Ders metni + Özet Çıkarımlar |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|

Modül kimlikleri kebab-case, sıfır dolgulu ve bir kez atandıktan sonra sabit. Proje ortasında asla yeniden adlandırılmıyorlar.

## Kod Stili

Dosya başına tek kavram. Yorumlar *nedenini* öğretiyor; asla sözdizimini anlatmıyor.

```
// Module 02 – examples/AutoboxingTrap.java
//
// Java has two parallel worlds for numbers: primitives (int) and objects
// (Integer). Most of the time Java quietly converts between them, which is
// convenient right up until the moment it bites you. Here is the bite.

void main() {
    Integer a = 127;
    Integer b = 127;
    IO.println(a == b);    // true – and this is the misleading one
}
```

Gelenekler: standart Java adlandırma ( `PascalCase` tipler, `camelCase` üyeler, `UPPER_SNAKE` sabitler), 4 boşluk girinti, 100 karakteri aşmayan satırlar, her örnek argümansız tek başına çalıştırılabilir.

---

## Test Stratejisi

---

Burada birim testi yapılacak bir uygulama yok; teslim edilen şey öğretim materyali. Doğruluk, **her örneğin gerçekten çalışması** demek.

- `scripts/verify-examples.sh`, `modules/**/examples/*.java` ve `solutions/**/*.java` dosyalarını gezer, her birini çalıştırır, sıfır olmayan her çıkışta ya da derleme hatasında başarısız olur.
  - Bilerek derleme hatası gösteren örnekler `// EXPECT: compile-error` başlığı taşır ve atlanmak yerine başarısız olmaları denetlenir.
  - Preview bayrağı gerektirenler `// EXPECT: preview` taşır ve `--enable-preview --source 27` ile çalıştırılır.
  - Bir modül, örnekleri, ödev metni ve referans çözümünü scriptten geçene kadar "bitti" sayılmaz.
  - Modül 21 testi `assert` ve elle yazılmış bir çatıyla öğretir, sıfır bağımlılık kuralını bozmadan, ve gerçek bir JUnit kurulumunun ne katacağını anlatır.
- 

## Sınırlar

---

### Her zaman

- Tüm ders metnini, kodu, yorumları ve tanımlayıcıları İngilizce yazın.
- Her modüle bir Özet Çıkarımlar bölümü ve bir ödev verin.
- Bir modülü bitmiş saymadan önce her örneği çalıştırın.
- Yaygın bir hatayı anlatırken öğrencinin göreceği gerçek hata mesajını gösterin.
- Bir özellik preview, inkübatör ya da JDK sürümüne bağlıysa bunu açıkça belirtin.

### Önce sorun

- JUnit dahil herhangi bir üçüncü parti bağımlılık eklemek.

- Maven ya da Gradle getirmek.
- Onaylanmış modül kimliklerini, sıralarını ya da müfredat taslağını değiştirmek.
- Kapsamı çatılara, Android'e ya da web geliştirmeye genişletmek.

## Asla

- Bilerek olduğu işaretlenmeden derlenmeyen bir örnek yayımlamayın.
- JEP numarası, yayın tarihi ya da özellik iddiası uydurmayın. [openjdk.org](https://openjdk.org) üzerinden doğrulayın.
- Tanımlamadan önce jargon kullanmayın.
- Kullanımdan kaldırılmış ya da önerilmeyen bir pratiği öyle olduğunu belirtmeden sunmayın.
- Ödev çözümlerini modül klasörünün içine koymayın.

## Başarı Kriterleri

Sekizinin hepsi 2026-09-23'te teslim edilen depoya karşı denetlendi. Her satır yalnızca sağlandığını değil, **nasıl doğrulandığını** kaydediyor.

- ✓ **1. `./scripts/verify-examples.sh` tüm depoda sıfırla çıkıyor.** 130 dosya, hepsi bildirdiği gibi davranıyor, yaklaşık 70 saniyede.
- ✓ **2. 22 modül klasörünün hepsi var, her biri `README.md`, en az iki çalıştırılabilir dosya içeren `examples/` ve `homework/README.md` içeriyor.** Script ile her modül dizininde denetlendi. Toplam 105 örnek dosya, modül başına en az 4.
- ✓ **3. Her modül README'si yedi şablon bölümünün hepsini, Özet Çıkarımlar dahil içeriyor.** 22 dosyanın her birinde başlıklar grep ile denetlendi. Son geçişte `22-whats-new` içinde bir eksik bulundu ve düzeltildi.
- ✓ **4. Her ödevin `solutions/<modül-kimliği>/` altında temiz çalışan eşleşen bir referans çözümü var.** 22 modül için 22 çözüm dizini, 25 çözüm dosyası, hepsi yukarıdaki doğrulama çalıştırmasına dahil.
- ✓ **5. Kök `README.md` her modülü sırayla bağlıyor ve JDK 27 kurulumunu belgeliyor.** 22 bağlantının hepsi script ile doğrulandı. Kurulum macOS, Linux ve Windows için, her biri `JAVA_HOME` talimatıyla belgelendi.
- [~] **6. Hiçbir dosya İngilizce olmayan metin, tanımlayıcı ya da yorum içermiyor. Ek 1 ile geçersiz kılındı.** İngilizce müfredat ve tüm kod için doğru, ki kriterin koruduğu şey buydu. Depo bütünü için artık doğru değil, çünkü `tr/` altında her dersin Türkçe çevirisi duruyor. Yerine 9 ile 12 arası kriterler geçti.
- ✓ **7. Depoda hiçbir yerde üçüncü parti bağımlılık yok.** `pom.xml`, `build.gradle` ya da `.jar` yok. 130 dosyadaki her import `java.*`, `javax.*` ya da `jdk.*` altında çözülüyor, artı bir bağımlılık değil dil özelliği olan JEP 511 `import module java.base` bildirimi.

**\*\*Ek 1 notu.\*\*** `scripts/build-pdfs.sh`, pandoc ve Chromium tabanlı bir tarayıcı gerektiriyor. Bunlar dokümantasyon üretmek için çağrılan dış araçlar\*, müfredatın kodunun bağlandığı kütüphaneler değil. Öğrencinin derlediği ya da çalıştırdığı hiçbir şey onlara bağlı değil, ve PDF'ler isteğe bağlı çıktı. Kriter Java kodu için yazıldığı gibi geçerli, ve bu not ayrımın varsayılmak yerine belirtilmesi için var.

- ✓ **8. Bir JDK sürümü hakkındaki her olgusal iddia openjdk.org yayın verisine dayanıyor.** Modül ve kök README'lerde 32 ayrı openjdk.org bağlantısı. JDK 26 ve 27 JEP tabloları, JDK 25 kesinleşme iddiaları ve JEP 491'in sanal iş parçacığı sabitletmesini kaldırması, yazım sırasında hatırlanmak yerine kaynağından alındı.

## Açık Sorular, Çözöldükleri Hâliyle

- JDK 27 kurulumu.** Çözöldü. Uygulama başlamadan önce JDK 27 (derleme 27+35-2325) kuruldu ve her örnek ona karşı doğrulandı.
- Modül 21 ve JUnit.** Sıfır bağımlılık kuralı lehine çözöldü. Test, çıplak `assert` artı 39 satırlık elle yazılmış bir çatıyla öğretiliyor, ve modül gerçek bir projede JUnit benimsemenin gerekçesini kuruyor. Kısıt dersi iyileştirdi: çalışan bir çatının tamamını görmek, bir framework'ün ne yaptığını onu kullanmaktan daha iyi açıklıyor.
- Preview özellikler.** Önerildiği gibi çözöldü. Yapılandırılmış eşzamanlılık (JEP 533) ve ilkel tip desenleri (JEP 532) yalnızca Modül 22'de, JEP numarası ve preview turuyla etiketli olarak, `--enable-preview` arkasında görünüyor. Modül preview özellik yayımlamaya açıkça karşı çıkıyor, çünkü bir preview sınıf dosyası bayrakla bile farklı bir sürüm tarafından reddediliyor.

## Uygulama Sırasında Yapılan Düzeltmeler

Kaydedildi, çünkü spesifikasyonun sınırı iddia uydurmayı yasaklıyordu ve buna bağlı kalmak yanlış olan birkaç şeyi açığa çıkardı:

- Bir kompozisyon örneği, `ArrayList` alt sınıflamanın eklemeleri iki kez sayacağını iddia ediyordu. Çalıştırmak bunu çürüttü: modern `ArrayList.addAll` toplu kopyalıyor. Hatayı gösteren koleksiyon `HashSet`, ve örnek artık ikisini de gösteriyor, ki bu kırılğan temel sınıf noktasını özgününün yapacağından daha iyi kuruyor.
- Alıntılanan üç derleyici hatası satır numarası yanlış. Hepsi hatalar üretilerek düzeltildi.
- Bir kova indeksi gösterimi 8 kova kullanıyordu, ve `Integer.MIN_VALUE` 8'e tam bölünüyor, dolayısıyla hem bozuk hem doğru biçim `0` döndürüyordu ve hata görünmezdi. 7'ye değiştirildi.
- Bir ödevin belirtilen çıktısı referans çözümünün yazdığınıyla çelişiyordu. Ödev çıktıları artık gözle değil çözüm çıktısına karşı `diff` ile karşılaştırılıyor.

- Yapılandırılmış eşzamanlılık preview geçmişi hafızadan yedi tur olarak yazılmıştı. JEP 533'ün History bölümünü okumak, öncesinde iki inkübatör turu daha olduğunu gösterdi, yani dokuz sürüm.

## Ek 1 — PDF çıktısı ve Türkçe çeviri

**Tarih:** 2026-09-23, özgün 24 görev teslim edildikten sonra.

Spesifikasyon onaylandıktan sonra iki yetenek eklendi, ve hiçbirini inşa edildiği sırada herhangi bir spesifikasyonda ya da görev listesinde yer almıyordu. `spec-driven-development` becerisi tam olarak bu durumu bir kırmızı bayrak olarak isimlendiriyor: "*Hiçbir spesifikasyonda ya da görev listesinde geçmeyen özellikleri uygulamak.*" Bu ek, dokümanı gerçekte teslim edilenle yeniden hizalıyor.

### Eklenen ne

| Yetenek                 | Ne olduğu                                                                                                                                                                          |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PDF çıktısı</b>      | <code>scripts/build-pdfs.sh</code> her Markdown dosyasını PDF'e, artı dil başına birleşik bir kitaba dönüştürüyor. <code>scripts/pdf.css</code> onları baskı için biçimlendiriyor. |
| <b>İki dilli içerik</b> | <code>tr/</code> altında 48 Markdown dosyasının tamamının Türkçe çevirisi, artı çeviri kurallarını kaydeden <code>tr/CEVIRI-NOTLARI.md</code> .                                    |

### Çevirinin bilerek çevirmediği

Burada kaydedildi çünkü bu bir tercih değil bir kısıt, ve özgün spesifikasyonun Sınırlar bölümünde ikinci bir dili kapsayan hiçbir kural yoktu:

- Kod blokları aynen İngilizce kalır.** `examples/` altındaki gerçek dosyaları alınıyorlar, dolayısıyla yorumlarını çevirmek öğrencinin gerçekten çalıştığı şeyle örtüşmeyi bozardı.
- Derleyici çıktısı aynen kalır.** Çevrilmiş bir hata mesajı aratılamaz ve terminalde yazılanla eşleşmez.
- Komutlar, dosya yolları, Java API adları ve JEP numaraları olduğu gibi kalır.**
- `tr/` altında hiç Java kaynağı yok.** Her iki dil aynı `modules/*/examples/` ve `solutions/` ağaçlarını paylaşıyor.

### Bu ekin yakaladığı bir ihlal

Yukarıdaki kuralı yazmak onu denetlemeyi zorunlu kıldı, ve denetim başarısız oldu. **On sekiz Türkçe dosyadaki otuz beş Java kod bloğunun yorumları çevrilmişti**, ki bu çevirinin kendi belgelediği kuralın doğrudan ihlali. İngilizce özgünlerine geri alındılar, ve 10 numaralı kriter artık tekrarına karşı koruyor.

Kuralı akılda tutmak yerine yazmanın değeri bu: kural açıktı, yazılmıştı, ve herhangi bir şey denetlemeden önce yine de kırk beş kez çiğnendi.

## Güncellenmiş başarı kriterleri

1 ile 5 arası, 7 ve 8 numaralı kriterler değişmeden geçerli. 6 numaralı kriter geçersiz kılındı.

- ✓ **9. Her İngilizce Markdown dosyasının `tr/` altında aynı yolda bir Türkçe karşılığı var.** 48'inin hepsi script ile doğrulandı.
- ✓ **10. Hiçbir Türkçe dosyadaki hiçbir `java` kod bloğu İngilizce olmayan metin içermiyor.** 35 tanesi bulunup geri alındıktan sonra sıfır ihlal.
- ✓ **11. Java kod blokları iki dil arasında birebir hizalı.** Eşleşen 44 dosyanın her birinde sayı aynı, dolayısıyla okuyucu her iki sürümü de aynı koda karşı takip edebiliyor.
- ✓ **12. `tr/` altında hiç Java kaynak dosyası yok.** Sıfır. Her iki dil tek bir örnek ve çözüm kümesini paylaşıyor, dolayısıyla doğrulama scripti ikisini birden kapsıyor.
- ✓ **13. `scripts/build-pdfs.sh` her iki dili de üretiyor.** `pdf/` altında 49 İngilizce PDF, `pdf-tr/` altında 50 Türkçe, her birinde birleşik bir kitap, yaklaşık 100 saniyede.
- ✓ **14. Çeviri örnek doğrulamasını etkilemiyor.** `./scripts/verify-examples.sh` hâlâ 130 dosyanın bildirdiği gibi davrandığını raporluyor.

## Süreç notu

Doğru sıra, her iki yeteneği de inşa etmeden **önce** bu spesifikasyonu güncellemek, sonra plan ve görevleri ondan türetmek olurdu. Bu olmadı: ikisi de doğrudan istek üzerine inşa edildi ve spesifikasyon bayat kaldı, 6 numaralı kriter yanlış olduğu hâlde `[x]` olarak duruyordu.

Eki yazmak, aksi hâlde yayımlanmış olacak kod bloğu ihlalini açığa çıkaran şey oldu. Ders, bu müfredatın Modül 21'inin testler hakkında söylediğiyle aynı: kimsenin denetlemediği bir kural zaten çığnemiş bir kuraldır.